

Заключение

В ходе работы были реализованы и исследованы структуры данных hash table и binary heap на Python и C++. Для каждой структуры были подобраны встроенные или библиотечные аналоги, после чего было проведено экспериментальное сравнение производительности на разных размерах входных данных.

Основная цель работы заключалась в том, чтобы показать, что теоретическая асимптотическая сложность не всегда полностью объясняет практическое время работы структуры данных. Эксперименты подтвердили, что реализации с одинаковыми или близкими оценками сложности могут существенно отличаться по скорости выполнения операций.

Для hash table были рассмотрены операции вставки, поиска и удаления элементов. Собственная реализация использовала открытое адресование и double hashing. Такая стратегия позволяет уменьшить эффект кластеризации и показывает более устойчивое поведение по сравнению с более простыми способами пробирования. Однако даже при хорошей асимптотике собственная реализация на Python заметно уступает встроенному dict. Это связано не с различием в теоретической сложности, а с тем, что dict реализован на низком уровне и содержит большое количество оптимизаций.

Сравнение hash table также показало, что C++-реализации имеют преимущество за счет меньших накладных расходов языка, компиляции в машинный код и более прямого управления памятью. Особенно показательным оказалось сравнение с boost::unordered_flat_map. Эта реализация использует более эффективную организацию данных в памяти и лучше задействует кэш процессора, что позволяет ей показывать высокую производительность на больших объемах данных.

Для binary heap были исследованы операции добавления элементов, извлечения корня и смешанные сценарии, характерные для priority queue. Теоретически вставка и удаление корня в бинарной куче выполняются за $O(\log N)$, а получение минимального элемента --- за $O(1)$. Эксперименты показали, что эта асимптотика хорошо описывает общий характер роста времени, однако не объясняет всех различий между реализациями.

Собственная реализация binary heap на Python уступает heapq, поскольку heapq является компактной и оптимизированной библиотечной реализацией. При этом обе реализации используют одну и ту же базовую идею хранения кучи в массиве. Различия возникают из-за стоимости Python-операций, накладных расходов на вызовы методов, особенностей доступа к объектам и уровня оптимизации конкретной реализации.

C++-реализация binary heap и std::priority_queue также имеют близкую теоретическую основу, но могут различаться по постоянным множителям. На практике важную роль играют детали реализации операций sift up и sift down, количество сравнений и обменов, а также то, насколько эффективно данные расположены в памяти.

Отдельное внимание в работе было уделено общим причинам различий производительности. Было показано, что на реальное время выполнения влияют язык программирования, модель выполнения программы, кэш-локальность, memory layout, constant factors, количество аллокаций, работа с объектами и внутренние оптимизации библиотек. Эти факторы не отражаются напрямую в асимптотической записи, но именно они часто определяют, какая реализация окажется быстрее на практике.

Проведенные benchmark-эксперименты включали несколько размеров входных данных и повторные запуски. Для обработки результатов использовались среднее значение, медиана, стандартное отклонение и стандартная ошибка среднего. Результаты были представлены в виде таблиц и графиков. Такой подход позволил не опираться на единичные измерения, а анализировать более устойчивую картину поведения реализаций.

Графики показали, что при увеличении размера входных данных различия между реализациями становятся более заметными. На малых размерах входных данных часть различий может скрываться из-за малой абсолютной величины времени выполнения, поэтому для анализа использовались увеличенные фрагменты графиков. Error bars в большинстве случаев почти не видны, что говорит о малом разбросе между повторными запусками и стабильности измерений.

Таким образом, поставленная цель работы была достигнута. Были реализованы структуры данных, проведено сравнение с библиотечными аналогами, построены таблицы и графики, а также проанализированы причины различий в производительности. Работа показала, что асимптотическая сложность является необходимым, но недостаточным инструментом для оценки эффективности структуры данных.

Главный вывод состоит в том, что при выборе структуры данных и ее реализации необходимо учитывать не только теоретическую сложность операций, но и практические факторы: язык программирования, особенности реализации, расположение данных в памяти, кэш процессора и качество библиотечных оптимизаций. Именно сочетание теоретического анализа и экспериментального измерения позволяет получить более полное представление о реальной эффективности структур данных.